Common runtime errors

Purpose

This page lists the messages SWAT+ 62.0.1 writes when a run stops or when it cannot match a name in the inputs, and the input problems that change the results without any message. For each one it gives the cause and the fix.

Message locations

Place	Content
the screen (standard output)	errors that stop the run, written just before it stops
`diagnostics.out`	names that could not be matched while the inputs were read; the run continues
`simulation.out`	the revision banner and one line per simulated day, ending with `Execution successfully completed`
`gwflow_record`	messages from the gwflow module, including the ones that stop the run

Messages printed on the screen are not copied to `simulation.out`. When the model is started by a script or another program, keep the screen output, for example with `swatplus > run.log 2>&1`. The layout of the log files is described in Checker and diagnostics.

In the messages below, `<name>` stands for a name from the inputs and `<n>` for a number. SWAT+ pads names with spaces, so the gaps in a real message are wider.

Errors that stop the run

Missing object.cnt

```
Cannot find object.cnt input file
```

The file named on the `simulation` line of `file.cio` (normally `object.cnt`) does not exist, or the name is `null`. Usually the model was started outside the project folder. Change into the folder that contains `file.cio` and start it again. Started in an empty folder, SWAT+ also leaves a few files behind (see Files the model creates).

Carbon input files

With `carbon = 2` in `codes.bsn`, two files are required. SWAT+ stops with one of these messages, on the screen and in `diagnostics.out`:

```
ERROR: <name> is required when carbon is enabled (codes.bsn carbon = 2)
ERROR: <name> could not be opened
ERROR: <name> data/values line is missing or could not be parsed (expected 28 values)
ERROR: <name> provided <n> row(s); expected <n>
```

The first file is the one named in the third slot of the `basin` line of `file.cio`, normally `carbon.bsn`. The second has no slot: its name is the first name with `_lyr` added before the extension, normally `carbon_lyr.bsn`. Provide `carbon.bsn` with its 28 values and `carbon_lyr.bsn` with a row for each layer group, or set `carbon` to 0. The old `carb_coefs.cbn` is no longer read.

If the message names a file that is not a carbon file, such as `ERROR: climate is required when carbon is enabled`, the `basin` line of `file.cio` has only two names. See Old-format inputs.

print.prt rows

```
Error: The output object <name> in the input file print.prt is not a valid object.
       and cannot be processed
Error: <name> print object is duplicated in the input file print.prt.  Aborting
```

These checks apply when `use_obj_lbls` is `y` in `print.prt`, so that the rows are read by name. A row name that SWAT+ does not know, or a row given twice, stops the run. The valid names, and the files each one opens, are listed in print.prt rows and files.

Output folder

```
 ! ERROR: cannot create path specified in file.cio (out_path)
   > Windows-style path detected on Unix system: <name>
 ! ERROR: cannot create or access path specified in file.cio (out_path)
   > Path: <name>
```

The optional output folder line of `file.cio` names a folder SWAT+ cannot use. On Linux and macOS a folder name that contains `:` is taken as a Windows path and refused. The second message means the folder could not be created or opened, often because of file permissions. See Output folder.

gwflow input files

The gwflow module writes its messages to `gwflow_record` in the run folder, so the screen shows only that the program stopped. These messages stop the run:

Message in `gwflow_record`	Cause
`ERROR: gwflow.cells row has too few columns at i=<n>`	row *n* of `cells.gw` has fewer than the 14 required columns
`ERROR: gwflow.cells row out of order at i=<n> cell_id=<n>`	the cell IDs in `cells.gw` are not 1, 2, 3, … in file order
`ERROR: gwflow.cellcons row has too few columns at i=<n>`	row *n* of `cellcon.gw` has fewer than 2 columns
`ERROR: gwflow.cellcons row out of order at i=<n> cell_id=<n>`	the cell IDs in `cellcon.gw` are not in file order
`neither gwflow.hrucell or gwflow.lsucell found; stop simulation`	`conn_type` in `codes.gw` is neither 1 nor 2

The messages use older file names: `gwflow.cells` is `cells.gw` and `gwflow.cellcons` is `cellcon.gw`. With `conn_type` 1 or 2 and neither `hrucell.gw` nor `lsucell.gw` present, the run does not stop. See gwflow inputs.

Fortran run-time errors

Some readers do not check what they read. A missing line, a missing file or text where a number is expected then stops the run with a message from the Fortran run-time library instead of SWAT+. With gfortran builds it contains `Fortran runtime error`, and with Intel builds it starts with `forrtl: severe`. The message usually gives the file name or the unit number. Check that file for a missing or extra line or a wrong number of columns, and compare it with its page in the Input Reference.

Most readers do check, and do not stop. When a value in a row cannot be read, they stop reading that row without a message, and the columns after it keep their defaults. A row with a stray character or a missing value can therefore change the results silently.

Messages in diagnostics.out

Line 1 of `diagnostics.out` is `DIAGNOSTICS.OUT FILE`. Each later line is written while the inputs are read, and the run continues. From a run of the `Ames_sub1` reference dataset:

```
agrr_lum                                AGRR_rot                                not found in
↪  management.sch
ames.tem                                        file not found (tgage)
```

Land use names

Written for a row of `landuse.lum` that names a record that is not found. The line gives the land use name, the missing name and one of these endings:

Ending	Column	Result
`not found in plant.ini`	`plnt_com`	no plants at the start
`not found in management.sch`	`mgt`	no scheduled operations: no planting, fertilizer or harvest
`not found in cntable.lum`	`cn2`	built-in curve numbers 30, 55, 70 and 77 for soil groups A to D
`not found (landuse-consprac)`	`cons_prac`	P factor 1.0
`not found (landuse-tiledr)`	`tile`	no tile drain
`not found (landuse-septic)`	`sep`	no septic system
`not found (landuse-fstrip)`	`vfs`	no filter strip
`not found (landuse-grassww)`	`grww`	no grassed waterway
`not found (landuse-bmpuser)`	`bmp`	no user BMP

Correct the name in `landuse.lum`, or add the missing record to the file it points to. See Name matching. `Ames_sub1` writes four `not found in management.sch` lines, so the HRUs with those land uses have no management.

Plant names

```
mgt schedule <n> op numb <n> <name> not found in plants.plt database
plant com <n> plant numb <n> <name> not found in plants.plt database
```

The first is written for a `plnt` line of `management.sch` whose plant name is not in `plants.plt`; the numbers are the position of the schedule in the file and of the line in the schedule. The second is written for a plant in `plant.ini` that is not in `plants.plt`. Plant names are case-sensitive.

Weather station names

```
<name> file not found (tgage)
```

A station in `weather-sta.cli` names a data file that is not listed in the matching index file. The ending gives the column: `(pgage)` precipitation and `pcp.cli`, `(tgage)` temperature and `tmp.cli`, `(sgage)` solar radiation, `(hgage)` humidity, `(wgage)` wind, `(petgage)` PET and `(atmodep)` atmospheric deposition. For the five weather variables, SWAT+ then generates that variable for the station on every day from its weather generator record. In `Ames_sub1`, `weather-sta.cli` names `ames.tem` while `tmp.cli` lists `ames.tmp`, so the temperatures are generated.

`file not found (wgn)` means the `wgn` name of the station is not in `weather-wgn.cli`. 62.0.1 has no record to fall back on in that case, and the generated weather of that station is undefined. Correct the name before running. See Unmatched names.

Reservoir, wetland and other names

Ending	Name that was not found
`not found (res-init)`, `(res-hyd)`, `(res-release)`, `(res-sed)`, `(res-nut)`	a record named in `reservoir.res`
`not found (wet-init)`, `(wet-hyd)`, `(wet-release)`, `(wet-sed)`, `(wet-nut)`	a record named in `wetland.wet`
`not found (wetland.wet)`	the `surf_stor` name of an HRU in `hru-data.hru`; the HRU has no surface storage
`not found (field.fld)`	the `field` name of an HRU; the HRU uses the built-in field size

Ending	Name that was not found
`not found (plants.plt)`	the plant of an HRU in `hru-lte.hru`
`entry in (hru-lte.hru) not found in (lum.dtl)`	a decision table named in `hru-lte.hru`
`not found (initial.cha)`, `(hydrology.cha)`, `(sediment.cha)`, `(nutrients.cha)`	a record named in the legacy `channel.cha`, which 62.0.1 reads but does not route

Carbon layer files

```
<name>: layer id <n> is out of range (1 to <n>); row ignored
carbon_layers.prt: n_layers must be >= 1; got <n>; default <n> kept
```

The first means a row of `carbon_lyr.bsn` has a layer number other than 1 or 2, and the row is skipped. The second means the optional layer count file has a value below 1, and the per-layer carbon outputs use the largest layer count of any HRU.

Names matched without a message

Several names are looked up without any report when they are not found. The run continues with the result in the table, so check these names by hand or in SWAT+ Editor.

Name	Result when it is not found
`topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init` or `snow` in `hru-data.hru`	the HRU uses the first record of the target file. The reader has messages for these names, but they are never written in 62.0.1.
`wst` in a connection file such as `hru.con`	the object uses the first station in `weather-sta.cli`. The stations must also be listed in ascending order of name for the lookup to work. See Weather station lookup.
fertilizer name of a `fert` line in `management.sch`	nothing is applied. The name is read from `op_data1`; see the management.sch example.
harvest operation of a `harv` line	nothing is harvested
tillage name of a `till` line	no soil is mixed
transplant name of a `plnt` line or a decision table plant action	the plant starts from seed. In 62.0.1 the file is `transplant.ops`, and a file named `transplant.plt` is not read. See transplant.ops.
manure name of a `manu` line	the manure is found by name in `manure_db.frt`, but its make-up comes from the row of `manure_om.frt` in the same position, whatever `org_min` names. See manure_om.frt.
decision table named on an automatic operation line of `management.sch`	the table is not applied. See Automatic operations.
`urban` or `ov_mann` in `landuse.lum`	the HRU is not urban, or keeps Manning's n of 0.05

Files the model creates

For a number of input files, 62.0.1 opens the named file without checking that it exists, and the operating system then creates an empty file of that name. The run continues as if the file had no records. This happens for `time.sim` and `print.prt`, the place-of-use and point-of-diversion files of water allocation, `wtps_wuses.wal`, and the initial-condition files for pesticides, pathogens, metals, salts and constituents (`pest_hru.ini`, `salt_hru.ini`, `cs_aqu.ini` and others). For some of these slots a `null` name creates a file named `null`.

An empty file left behind changes the next run: an empty `time.sim`, for example, leaves the run without a valid simulation period. After a run that misbehaves, list the empty files in the project folder (`find . -size 0` on Linux and macOS) and restore or delete them. If in doubt, write the inputs again from SWAT+ Editor.

Old-format inputs

Files written for an earlier revision are read without a version check. These changes matter most for 62.0.1:

- **file.cio `basin` line.** It now has three names, the third being the carbon file. With only two, every later line of `file.cio` is read one line too low and all later file names are wrong, with no message. With `carbon = 2` the run stops with `ERROR: climate is required when carbon is enabled`. See Older file.cio files.
- **file.cio `water_rights` line.** It now has two names. An old line with `water_allocation.wro element.wro water_rights.wro` makes SWAT+ read `water_allocation.wro` as the place-of-use file.
- **transplant.plt.** Renamed to `transplant.ops`. A project that still has `transplant.plt` plants from seed.
- **pesticide.pes.** Three columns were added after the existing 13 values. Older rows are still read, and the new columns keep their defaults, which means no plant uptake. See pesticide.pes.
- **print.prt.** A file with `dbout` in place of `use_obj_lbls` is read by position, so an extra or missing row moves the switches of every later row to the wrong output group. See print.prt.
- **carb_coefs.cbn.** No longer read. Its settings are in `carbon.bsn` and `carbon_lyr.bsn`.

Most input files start with a title line and a column header line, and SWAT+ skips both. A file that starts with the header loses its first data row, with no message. In the reference data, `tillage.til` of `Ames_sub1` and `puddle.ops` of `Osu_1hru` have no title line, so their first entries are not read; see tillage.til and puddle.ops.

Runs that stop without a message

Full disk. When the disk fills, the run stops with a Fortran run-time error about writing a file, or with no message at all, and the last output files are cut short. Other programs that use the same disk start to fail as well. Daily output is the usual cause: a full 46-year run of the `Ames_sub1` reference dataset writes about 9 GB, almost all of it the daily carbon files switched on by its `hru_cb` and `hru_cb_vars` rows. Check the free space before long runs (`df -h .` on Linux and macOS) and switch off the daily outputs you do not need; see Output volume.

The last day. The last daily line in `simulation.out` is the last day that finished, so the problem occurred on the next day. Climate records that are shorter than the simulation do not stop the run: SWAT+ generates the missing days (see Weather outside the observed record).

A run that never gets past its first day. Earlier revisions could repeat the operations of a management schedule without end when every operation had the same month and day. 62.0.1 runs each operation at most once per pass on a given day, so a run that hangs at the start with 62.0.1 has another cause. A debug build with run-time checks usually stops with a message that points to it.

Related files

- Checker and diagnostics: the layout of `simulation.out`, `diagnostics.out`, `checker.out` and `files_out.out`.
- Water balance issues: runs that complete but give implausible results.
- Install and Quick start with Ames_sub1: first runs, `Permission denied` and the start-up banner.

Sources

- Stops: `basin_read_objs.f90` (`object.cnt`), `carbon_bsn_read.f90`, `basin_print_codes_read.f90` and `print_prt_error` in `basin_module.f90`, `output_path_module.f90`, `gwflow_read.f90`
- `diagnostics.out` is opened as unit 9001 in `proc_bsn.f90`. Writers: `landuse_read.f90`, `read_mgtops.f90`, `readpcom.f90`, `cli_staread.f90`, `hru_read.f90`, `hru_lte_read.f90`, `res_read.f90`, `wet_initial.f90`, `ch_read.f90`, `carbon_layers_read.f90`
- Weather fallback: `climate_control.f90`, `cli_precip_control.f90`, `cli_bounds_check.f90`, `cli_wgnread.f90`
- Same-date guard: `mgt_operatn.f90`